

Cricket Analytics Project Documentation

Freshman Guide & Technical Architecture Blueprint

1. Project Overview & Tech Stack

This project is a premium, full-stack Cricket Analytics Dashboard designed to visualize batsman metrics, bowler matchups, scoring phases, and detailed dismissal histories. It uses real IPL data (from 2008 to 2026) harvested from the web.

Technologies Used:

- Front-End: Next.js (React), Tailwind CSS, Recharts (for visual rendering)
- Back-End: Python 3, Flask REST API (serves data as JSON)
- Data Analytics: Pandas, NumPy (cleanses and aggregates data in memory)
- Scraper: Requests & BeautifulSoup (extracts ball-by-ball commentary)

2. Project Folder Structure

Below is the clean directory layout of the project codebases:

```
cricket-app/  
|-- cricbuzz scraping/          # Tier 1: Data Scrapers & Match Lists  
|   |-- get_ipl_match_ids.py    # - Gathers IPL match IDs from Cricbuzz  
|   |-- scrape_2026_season.py   # - Extracts ball-by-ball commentary text  
|-- dataset/                   # Tier 2: CSV Database (2008-2026)  
|   |-- 2008.csv ... 2025.csv   # - Historical IPL seasons data  
|   |-- 2026.csv                # - Freshly scraped 2026 IPL logs  
|-- backend/                   # Tier 3: Python API Server  
|   |-- app.py                  # - Flask server endpoints (/api/seasons, /api/players)  
|   |-- data_processor.py       # - Data-cleansing & aggregation engine (Pandas)  
\-- frontend/                  # Tier 4: Next.js Web App UI  
    |-- src/app/page.js         # - Parent view controller & coordinator  
    \-- src/components/        # - Modular presentational UI widgets  
        |-- Header.jsx         # - Search bar & profile component  
        |-- Sidebar.jsx        # - Left filter menus & season controls  
        |-- StatsGrid.jsx      # - Striker KPI summary cards  
        |-- RecentMatches.jsx  # - Match logs list table  
        \-- DashboardInsights.jsx # - Strengths/Weaknesses insights container  
            |-- OversChart.jsx # - Runs per over bar chart  
            \-- DismissalsPieChart.jsx # - Wicket type donut chart (contains Log Modal)
```

3. Workflow & System Architecture Diagram

Here is the non-clustered, tiered workflow mapping how data flows from web sources to UI components:

```
[ Cricbuzz API Server (cricbuzz.com/api/mcenter) ]  
|  
v (HTTP GET requests)  
[ scrape_2026_season.py Scraper ]  
|  
v (Extracts, classifies & saves)  
[ dataset/ 2026.csv ]  
|  
v (Reads & Cleanses)  
[ data_processor.py (Pandas Engine) ]
```

Cricket Analytics Project Documentation

Freshman Guide & Technical Architecture Blueprint



Cricket Analytics Project Documentation

Freshman Guide & Technical Architecture Blueprint

4. Data Scraping & Cleansing Pipeline

1. Scraping Source:

We scrape Cricbuzz commentary REST APIs using Python's requests library. We attach realistic browser User-Agent headers to request raw JSON lists representing ball-by-ball descriptions.

2. Chronology Correction:

Cricbuzz sends the feed backward (newest ball first). The scraper reverses it to process balls chronologically.

3. Data Cleansing:

- Column Standardization: Maps diverse headers over seasons (2008-2026) to a single standard dataframe.
- Wicket Corrections: Rectifies raw anomalies (e.g. non-run-out wickets marked with runs are forced to 0).
- Name Normalization: Maps full strings ('Virat Kohli') to codes ('V Kohli') for uniform database grouping.
- Extras Identification: Classifies Wide (WD), No Ball (NB), Leg Bye (LB), and Bye (BY) from commentary text.

4. Feature Extraction (Text Mining):

Scans text using keywords to classify ball length (Yorker, Good Length, etc.), line (Outside Off, Stumps, etc.), shot type played (Drive, Sweep, etc.), and fielding catch position (Long-On, Point, Keeper, etc.).

5. Recent Architectural Modifications

The following changes were made recently to streamline state management and UI performance:

- Dismissal Log Modal Relocation:

Previously, the Dismissal Log Modal was triggered from the RecentMatches table. We have unlinked it from RecentMatches and moved it entirely inside DismissalsPieChart.jsx.

- Why this change was made:

1. Cohesive UI: The dismissal logs detail *how* a player got out (e.g. ball line/length and shot type). Grouping this next to the Wicket Donut Chart makes visual sense.
2. Isolated State: The state variable tracking modal visibility (showLogModal) now lives locally inside the Pie Chart component. The parent page.js component is no longer forced to handle modal states.
3. Cleaner Table: RecentMatches.jsx is now simpler and only displays general match performance figures.